NÜWA: A Generative Control Plane for AI Network Simulation

Wenkai Li
Xi'an Jiaotong University

Ran Shu
Microsoft Research Asia

Peng Zhang
Xi'an Jiaotong University

Yiren Zhao
University of Toronto

Danfeng Shan
Xi'an Jiaotong University

Yongqiang Xiong
Microsoft Research Asia

Abstract

Network simulation plays a critical role in improving the efficiency of large-scale AI clusters for design validation, parameter tuning, and protocol development. However, high-fidelity network simulation becomes prohibitively slow at scale, especially when running large batches of experiments on topologies with tens or hundreds of thousands of accelerators. We observe that a key bottleneck comes from the control plane. Existing network simulators typically compute routes and install forwarding tables at initialization, which can consume hundreds of GB of memory before packet-event execution begins and limit overall simulation throughput. In this paper, we present NÜWA, which views routing as a compilation problem, it leverages the hierarchical and symmetric structure common in AI fabrics and compiles a declarative topology description together with routing policies into compact forwarding artifacts that are fast to generate and efficient to look up. Evaluations show that NÜWA can reduce simulation initialization time from hours to only 25 seconds for a 65,536-GPU cluster. For end-to-end simulation time, NÜWA takes only 20% of that required by existing approaches in a 40K+ GPU cluster, and NÜWA can scale to a 221,184-GPU cluster.

CCS Concepts

• **Networks** → **Network simulations**; **Data center networks**; • **Computing methodologies** → **Discrete-event simulation**; **Simulation tools**.

Keywords

Network Simulation, Control Plane, AI Infrastructure, Data Center Networks

ACM Reference Format:

Wenkai Li, Ran Shu, Peng Zhang, Yiren Zhao, Danfeng Shan, and Yongqiang Xiong. 2026. NÜWA: A Generative Control Plane for AI Network Simulation. In *ACM SIGCOMM 2026 Conference (SIGCOMM '26), August 17–21, 2026, Denver, CO, USA*. ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3789240.3829106

1 Introduction

To support the rapid evolution of Artificial Intelligence (AI), companies are building AI clusters at extremely large scale [47]. For example, xAI trained Grok4 using a 200,000 GPU cluster [66]. To improve the performance of AI training, operators need to optimize parallelization strategies [27], tune network parameters [24], and develop new transport layer protocols [48]. However, this can be prohibitively expensive (e.g., running a 10K-GPU cluster costs over 1 million dollars per day [19]).

Simulation has therefore become a crucial tool for evaluating AI cluster designs and training stacks [27, 52, 63–65]. Since communication behavior directly impacts end-to-end AI training time, network simulation plays a key role in high-fidelity AI training simulation [27, 63]. Discrete-event simulation (DES) is often preferred to achieve such high accuracy, as it models packet-level behaviors precisely. Representative DES frameworks include ns-3 [46], OMNeT++ [41], and htsim [16].

For a long time, network DES research has primarily focused on the data plane, e.g., accelerating event processing to reduce the time of a *single* simulation run [13, 25, 34, 39, 63, 69, 73, 74]. However, in high-throughput simulation workloads for AI training (e.g., protocol hyperparameter tuning [27], large design space exploration [27, 67], and what-if analysis [37]), the overall throughput increasingly depends on executing *multiple* runs (e.g., about 10K experiments for large-scale AI training clusters [27]). In such settings, simulation throughput (i.e., completed simulation jobs per unit time on a fixed cluster) is not just limited by data-plane event processing, but also by the route computation and lookup in the *control plane*. When topologies approach or exceed 100,000 accelerators, which is a typical size of modern AI training infrastructures [2–6], the simulation can even run out of memory during control-plane processing (as detailed in §2). In our preliminary evaluation, on a Fat-tree [11] with 27,648 hosts, even when simple shortest-path routing is used, the initialization peak memory can reach nearly 200 GB. Without taming the control-plane bottleneck, the throughput of simulators will be fundamentally limited for large-batch, large-topology workloads, even when using the most advanced data-plane acceleration techniques.

A straightforward way to bypass the control-plane bottleneck is to use *handwritten* routing logic for specific topology families, an approach that has been adopted by several simulators [1, 16, 33, 38]. However, the efficiency gain comes at a cost in generality. First, existing handwritten schemes typically support only a small set of topology types. Extending support to a new topology requires implementing topology-specific routing logic, incurring substantial engineering effort. This limitation is exacerbated in the use case of design space search, where a large space of candidate topologies is simulated to find the optimal one for a given workload and/or objective (e.g., performance, cost) [27, 67]. Second, handwritten control-plane logic is inherently brittle in failure scenarios, which is an important concern of some simulation workloads. Handwritten approaches heavily rely on the assumption that the topology is regular, such that routes can be computed by reasoning over a



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
SIGCOMM '26, Denver, CO, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2467-1/26/08
https://doi.org/10.1145/3789240.3829106

limited number of switch roles or structural cases, while failures break this regularity in random ways. Consequently, one has to account for an enormous number of distinct failure scenarios, even when the failure rate is low (say 10^{-3}). In addition, non-minimal routing is also frequently used in simulation, which handwritten schemes cannot support.

Given the above tension between generality and scalability in the control plane of network simulators, we ask the following research question: *can we design a control-plane abstraction for mainstream network simulators that is both general and scalable for large-scale AI network simulation?*

In this paper, we propose NüWA, a generative [1] control plane for AI network simulation. The key idea of NüWA is to view the control plane construction as a compilation problem: given a topology specification, NüWA compiles it into compact routing artifacts, including shortest-path routing rules, policy-based non-minimal routing rules, and link failure handlers, rather than computing and storing a huge number of routes with routing or search algorithms. This can significantly reduce both memory and time costs. Then, forwarding decisions can be made efficiently based on the artifacts, instead of looking up entries in a large routing table, further improving the throughput of packet event processing.

Contributions. We make the following contributions:

- To the best of our knowledge, we are the first to study the control-plane scalability problem in DES-based network simulation for large-scale AI training, with detailed profiling results.
- We introduce NüWA, a new control-plane abstraction that treats routing as a *compilation* problem: given a declarative *topology specification*, NüWA compiles it into compact forwarding artifacts for shortest-path routing, policy-based non-minimal routing, and failover routing. This fundamentally avoids the expensive computation and storage of routes.
- We implement NüWA in ns-3 and show that it can reduce the control-plane memory footprint and initialization time by several orders of magnitude, while preserving control-plane correctness under shortest-path, non-minimal, and failure-aware routing.

Our implementation of NüWA is open source[2]. This work does not raise any ethical issues.

2 Background and Motivation

2.1 Network Simulation in the AI Era

Modern AI training clusters expose a vast and rapidly evolving design space across multiple layers, ranging from scale-out topology and routing to transport protocols, collective implementations, and job placement policies [17, 24, 51, 55, 62]. AI network simulation must extend beyond small-scale testbeds. At scale, networks exhibit emergent behaviors, such as correlated congestion, path diversity effects, and failure interactions, that are muted or absent in small topologies [57]. Evaluating such combinations directly on production clusters is costly, and often infeasible, especially when the target topology or protocol is not yet deployed or even implemented. Consequently, simulation becomes an indispensable tool for exploring AI network designs in a controlled, reproducible, and cost-effective manner [27, 56, 63, 65, 67].

Network simulation is a core component of end-to-end AI simulators used for system-level studies, where researchers and practitioners routinely run large batches of experiments for parameter sweeps, design-space search, and what-if analyses [22, 27, 67]. Such workflows demand both high throughput to finish many runs quickly and high fidelity to reflect behaviors under realistic routing, load balancing, and failures. These requirements bring renewed attention to a component that has been comparatively under-emphasized: the control plane of network simulation.

2.2 Control Plane in Network Simulation

Network model. A packet-level network simulator consists of a *data plane* model and a *control plane* model. The data plane models packet transmission and queuing behaviors. In contrast, the control plane computes end-to-end routing paths and populates forwarding tables that will be used to forward packets in the data plane.

Simulation stage. Simulation typically consists of two consecutive stages: the *initialization* stage and the *execution* stage. During initialization, the simulator constructs the topology, assigns addresses, computes end-to-end routing paths, and populates forwarding tables, which are mainly *control plane* tasks. During execution, it executes network events (e.g., packet sending, receiving, queuing, and forwarding) chronologically across a timeline, which are mainly *data plane* tasks.

The control plane is critical for AI network simulation. Previously, much optimization effort has focused on accelerating the *data plane* to handle massive numbers of packet events efficiently. As these optimizations mature [13, 25, 34, 50, 74], simulation throughput becomes increasingly dominated by control plane costs, including route computation and forwarding table construction across repeated runs. This shift is particularly pronounced in AI network studies, where large experimental grids require initializing and re-initializing routing state many times.

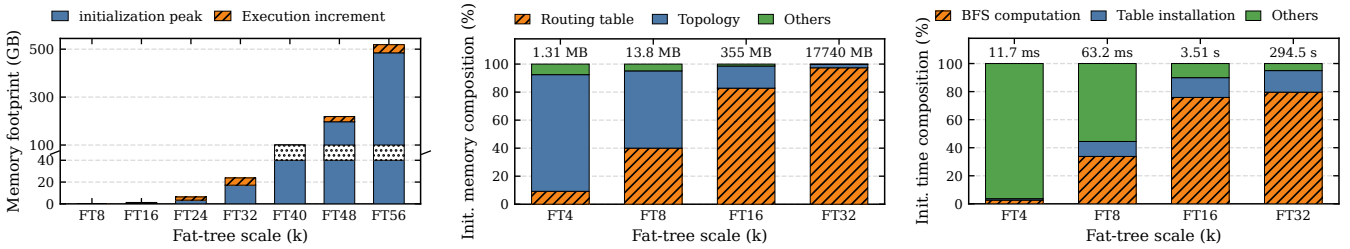
Beyond throughput, the control plane is critical for fidelity. First, AI clusters must tolerate realistic link and node failures while maintaining reachability and reasonable performance. Second, they also increasingly rely on non-minimal routing to exploit path diversity for load balancing and congestion avoidance. If the simulator's control plane cannot represent such policies or respond to failures in a realistic manner, the resulting simulated behaviors can deviate substantially from real systems, undermining the validity of conclusions drawn from simulation.

2.3 Limitations of Existing Approaches

Computing and storing per-destination forwarding tables A common approach is to compute global end-to-end paths via routing protocols [45, 53], shortest-path computations [20, 21, 71], or search-based algorithms (e.g., BFS), and then materialize forwarding tables for all switches. This design is widely adopted by mainstream network DES tools [12, 41, 46] because it is topology-agnostic and can respond to arbitrary failures.

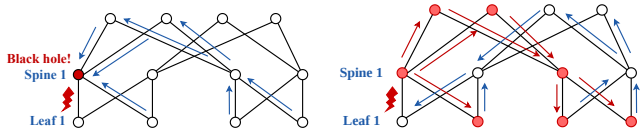
[1] Here, "generative" means that NüWA generates forwarding artifacts from declarative topology specifications which include both the topology structure and routing policies; it does not refer to generative AI.

[2] https://github.com/wenkai-tartar/NVWA-for-ns-3



(a) Memory footprint across simulation stages. **(b) Breakdown of initialization memory.** **(c) Breakdown of initialization time.**

Figure 1: Control-plane overhead during ns-3 simulation with an optimized BFS-based routing implementation. The vanilla ns-3 simulates OSPF [45] route computation and incurs even higher initialization overhead.



(a) Handwritten failure handling. **(b) NÜWA failure.**

Figure 2: Failure example in a Fat-tree simulation. (a) Disabling the failed port from Spine 1 to Leaf 1 is insufficient for topology-specialized handwritten routing. Packets may still follow the structural rule toward Spine 1 and reach a black hole. (b) NÜWA propagates the failure only to nodes whose forwarding choices are affected. The red nodes denote switches whose routing state changes after the failure. Each of them disables the corresponding ports for reaching the destination region under Leaf 1.

Table 1: Comparison of routing approaches.

	High Perf.	Topo Generality	Failure Handling	Policy Control	Scalable
Handwritten	✓	✗	✗	✓	✓
Protocols/ SP & Search	✗	✓	✓	✗	✗
NÜWA	✓	✓	✓	✓	✓

Protocols denotes routing protocols, SP means shortest-path routing, Search represents search-based algorithms.

However, computing and materializing routes at large scale can be prohibitively expensive. The control plane memory footprint and setup time grow quickly with network size, and the cost compounds across the many runs required for design space exploration. In practice, this can turn into control-plane computation that takes hours to complete and hundreds of GB of memory to store forwarding tables per run at scale, making the simulator unsuitable for production-grade iterative workflows.

Figure 1a separates memory usage by simulation stage in ns-3 simulations for different Fat-tree scales: the peak reached during initialization, called *initialization peak memory*, and the maximum additional memory used during execution, called *execution incremental memory*. The simulator reaches 197 GB at FT48 and 485 GB at FT56 during initialization. In contrast, the lightweight AllReduce workload adds much less memory during execution: at FT56, initialization peak memory is 6.8× the execution incremental memory.

Figure 1b further breaks down initialization memory into topology, routing table, and other simulator state. As the Fat-tree scale grows, routing table quickly becomes the dominant component, increasing from 6% at FT4 to 81% at FT16 and 97% at FT32.

Beyond memory footprint, the same control-plane setup also takes substantial time at hyper-scale. For heavy workloads, such as a full training iteration, execution time can still be the dominant cost. However, a 100K-node topology contains nearly 10^{10} source-destination pairs; even in an unweighted topology, computing all-pairs shortest paths requires running BFS from every source node and materializing the resulting forwarding state. Figure 1c further shows that as the topology grows, route computation and forwarding-table installation become the dominant parts of initialization time. This means the memory bottleneck caused by materialized routing state also translates into a time overhead, because the simulator must allocate, populate, and install large routing tables before execution starts. In our evaluation, FT56 takes more than 2 hours to finish initialization, while it takes less than an hour to finish execution for AllReduce using single-threaded ns-3.

Handling failure scenarios further exacerbates the cost. Injecting failures before a run is simply a one-time computation, while injecting failures dynamically during a run can trigger repeated recomputation that is even less tolerable.

Cached routing state. One might amortize initialization by storing the computed routing state and reusing it across experiments on the same topology. However, this replaces route computation with loading a massive routing-state snapshot. At hyper-scale, a snapshot for a topology can reach hundreds of GB to 1 TB, making reading, deserialization, and reconstruction of lookup structures expensive before execution begins. Moreover, reuse only mitigates repeated computation; it does not reduce the in-memory routing state required during simulation.

Distributed simulation. Distributed simulation helps scale packet-event execution by spreading events across multiple hosts [13, 25], and distributing simulation across multiple hosts has been shown to be effective for reducing memory pressure [60]. Partitioning the state reduces per-worker memory, but it introduces remote forwarding lookups and synchronization overhead on the packet-processing path. Replicating the routing state on every worker preserves local lookup but fails to reduce single-machine memory pressure. Therefore, distribution mitigates data-plane execution pressure, but the control-plane state explosion remains.

Topology-specialized handwritten routing logic. On the other hand, many simulators [1, 16, 33, 35, 38] adopt topology-specialized

handwritten routing logic that directly maps destinations (and sometimes policies) to next hops based on structural properties of specific topologies (e.g., Clos variants). This approach is fast and memory-efficient because it avoids global route computation and large forwarding tables.

The key limitation lies in generality. Supporting a new topology family, a new variant, or a new routing policy often requires substantial engineering effort to implement new hand-crafted logic. As a result, such simulators typically support only a small set of built-in topologies and are slow to adapt to emerging AI-network designs.

More importantly, handwritten logic tends to be brittle under failures. Locally disabling the failed port may not be sufficient to preserve consistent end-to-end reachability, potentially creating black holes where packets are forwarded toward an unavailable segment without a valid detour. Figure 2 illustrates such a failure-induced black-hole scenario in a handwritten Fat-tree simulation.

Therefore, existing control-plane designs embody a fundamental tradeoff. Computing per-destination forwarding tables offers generality and robust failure support, but suffers from high control-plane cost and poor scalability in time and memory. Handwritten routing is fast and lightweight, but lacks topology generality and struggles to handle failures and rich policy control in a realistic and maintainable manner. Table 1 summarizes these tradeoffs along five dimensions that matter for AI-network simulation.

These observations motivate our design goals for a control plane that is simultaneously fast, scalable, general across structured AI topologies, and faithful under failures and non-minimal routing.

2.4 Design Goals

High performance and scalability. A large number of simulation tasks are needed to explore optimal parameter settings, find the best topology structure for a specific workload, etc. This poses two challenges: (i) performance, where a simulation task should complete within hours; and (ii) scalability, where memory overhead should be small enough to simulate large-scale networks or run multiple simulation tasks simultaneously [27].

Topology generality for structured AI networks. AI network topologies are evolving rapidly, and new topologies are introduced at a very fast pace [44, 49, 61]. Moreover, simulators are being used to search the network topology design space [68]. To enable new topology exploration, the control plane should be able to generalize to new topologies. Specialized designs struggle to support such broad topology exploration.

Efficient and correct failure handling. Evaluating network performance under failures (static or dynamic) is a common task in AI infrastructure simulation and often incurs control-plane recomputation. Supporting failures also challenges generality because failures break the symmetry that topology-dependent optimizations often rely on, making a general and efficient failure-update mechanism necessary.

Support policy-based non-minimal routing. AI clusters increasingly seek to exploit path diversity (and sometimes more aggressive multipath) for load balancing and congestion avoidance [18, 43]. Modern fabrics also employ adaptive non-minimal routing to better balance load under unfavorable traffic patterns [54]. Thus, the control plane should expose policy controls to explore non-minimal routing that leverages path diversity for load balancing and congestion avoidance.

3 NÜWA Overview

3.1 Observations and Ideas

Observation 1: The default shortest-path forwarding behavior depends on how topologies are composed. Datacenter topologies are highly structured and are built by recursively interconnecting smaller topologies as building blocks. As we will show in §4.1, such composition can be *fully* specified by (i) the choice of interconnection points, i.e., which nodes in a building block are exposed for connections to other (higher-tier or lower-tier) nodes; and (ii) the wiring rules, i.e., how these nodes are wired together. Different connection-point choices and wiring rules induce different topologies and corresponding shortest-path forwarding behaviors.

Idea 1: Shortest-path forwarding can be resolved for different topologies in a uniform way. Rather than manually specifying shortest-path forwarding rules for each topology case by case, we can resolve the shortest-path forwarding behaviors for any topology based on its composition description file, which uses parameterized templates. We show that only a few patterns are needed to describe most of the datacenter topologies we are aware of.

Observation 2: The non-default policy-based forwarding behaviors are locally determined by shortest-path forwarding. Non-minimal routing in structured topologies is typically policy-driven. At a particular interconnection tier, the policy selects one or more intermediate nodes (waypoints). An end-to-end path is thus an ordered sequence from the source, through the waypoints, to the destination. The path can be decomposed into multiple segments between adjacent elements in the sequence. Within each segment, forwarding still follows shortest-path regularities. Segment-internal forwarding is determined by the relative structural relationship between the current node and the segment's target, yielding a fixed set of feasible egress ports. Thus, policy affects path shape primarily through waypoint selection, while the mechanism that generates each segment remains shortest-path.

Idea 2: Decompose the policy-based forwarding into a sequence of shortest-path forwarding. We decompose policy-based forwarding into two components: the policy component outputs a waypoint sequence to constrain the end-to-end path shape; the intra-segment forwarding directly reuses the shortest-path decision framework instantiated by Idea 1, together with its locally generated forwarding artifacts, to forward between successive points.

Observation 3: Failover forwarding behaviors are not common. After failures, the shortest-path and/or policy-based forwarding may be overridden. However, most of the time, the majority of the network is in a healthy state without failures, with a failure probability below 10^{-3} [70]. This means failures can be handled as exceptions, for which special treatment can be introduced with little impact on the overall simulation performance.

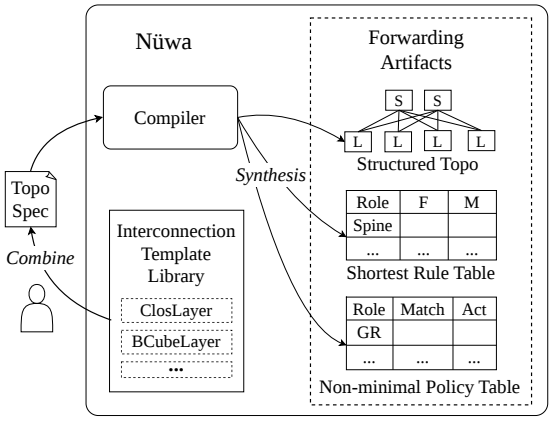
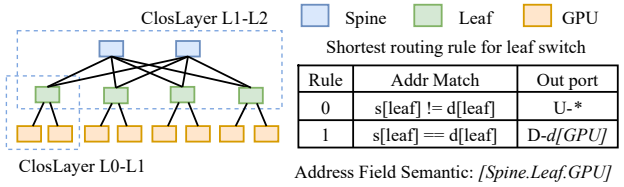


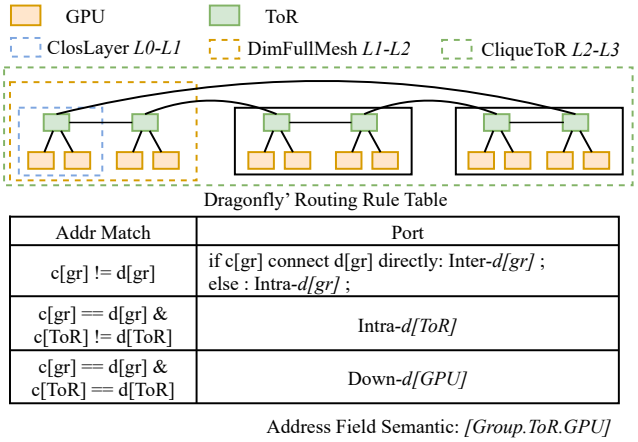
Figure 3: Nüwa architecture.

Rule	Params	Used in
ClosLayer	`group_num`	Fat-tree, Clos, HPN, RailOptimized, UB-Mesh, DragonFly(+)
BCubeLayer	–	BCube, ZCube
ZCubeLayer	–	ZCube
CliqueLF	`mode ∈ {abs,rel}`	DragonFly+
CliqueToR	`mode ∈ {abs,rel}`	DragonFly
DimMesh	`dim`	Mesh
DimFullMesh	`dim`	FullMesh, UB-Mesh
DimTorus	`dim`	Torus

Table 2: Interconnection template library. An illustrated reference is shown in Table 3.



(a) Leaf-spine topology.



(b) Dragonfly topology.

Figure 4: Topology construction examples.

Idea 3: Treat failures as exceptions. Based on Observation 3, failures introduce *exception cases* that deviate from the original symmetric shortest-path regularities. To handle these deviations without perturbing the normal case, we need an explicit exception representation that overrides the baseline rules only for failure-affected destination sets. We therefore introduce an exception mechanism that records failure-induced deviations and maintains them as per-node exception state. Nodes update this state by disseminating failure information throughout the network, enabling runtime forwarding to apply exception choices when needed and otherwise preserve the original regularities, thereby mitigating failure-induced asymmetry.

3.2 Architecture

Figure 3 shows Nüwa's architecture. Nüwa consists of a built-in *interconnection template library* and a *compiler*. The compiler compiles a high-level topology specification (Topo spec) into forwarding artifacts.

Nüwa takes as input a topology specification (Topo spec), which describes the target topology through interconnection templates selected from the interconnection template library (e.g., ClosLayer, CliqueToR), specifies the concrete parameters of these templates, and specifies their bottom-up application order.

The compiler first instantiates and composes interconnection templates in the order and with the parameters specified by the topology specification, constructing the topology's nodes, links, and connectivity and assigning structured addresses. It then synthesizes, over the instantiated topology, the templates' predefined forwarding semantics into unified pipeline-executable table entries and mappings.

Nüwa's output contains (i) a concrete structured topology instance whose nodes are each assigned a structured address; (ii) forwarding artifacts used to drive data-plane forwarding: a shortest-path routing rule table, a non-minimal routing policy table, and a port mapping table for each node.

4 Design Details

4.1 Interconnection Template Library

In structured AI fabrics, the forwarding decision often depends on the destination node's structural position rather than its individual identity. For example, from a leaf switch in a Clos layer, all destination nodes outside the local leaf unit are forwarded upward; their individual identities do not matter until later hops. Computing and storing one next-hop entry for each such destination therefore repeats the same structural decision many times. Nüwa addresses this inefficiency by explicitly modeling the relation between topology composition and the forwarding rules it induces.

Nüwa provides a *built-in interconnection template library* (shown in Table 2). Its templates describe how multiple topology units are composed into a larger topology unit, while simultaneously specifying the shortest-path forwarding semantics induced by the

composition. Conceptually, each template must define (i) a *structural layer* that selects the participating node roles, introduces new roles if needed, and wires nodes via links with ports grouped into named port buckets; and (ii) a *routing layer* that assigns structured addresses consistent with the new hierarchy and exports shortest-path routing rules expressed over these addresses. This separation allows templates to be composed bottom-up while keeping their induced forwarding logic explicit and reusable.

4.1.1 Interfaces. To make the above two-layer specification composable and compiler-friendly, each interconnection template exports a set of interfaces. The structural layer is realized by the *node role interface* and *wiring interface*. The routing layer is realized by the *address assigning interface* and *shortest rule interface*.

Node role interface. At each composition step, this interface: (i) selects which roles from the composed units act as interconnection endpoints, and (ii) specifies whether the template introduces any new roles in the resulting composed unit. These two aspects are illustrated by CliqueToR and ClosLayer in Figure 4. When composing level L1–L2 in Figure 4a, ClosLayer selects the *leaf* role as the interconnection endpoint (from each composed unit) and introduces a new role, *spine*. When composing level L2–L3 in Figure 4b, CliqueToR selects the *ToR* role as the interconnection endpoint and introduces no new roles.

Wiring Interface. This interface defines which node pairs among the selected node roles are connected through links and groups the ports into port buckets. For example, in Figure 4a, ClosLayer at level L1–L2 fully interconnects the *leaf* nodes of all composed units with the newly introduced *spine* nodes, and the connected ports of *leaf* nodes are grouped into an *up* port bucket. Whereas in Figure 4b, CliqueToR at level L2–L3 interconnects the *ToR* nodes across composed units, and the connected ports are grouped into an *inter* port bucket.

Address assigning interface. The structured address assignment of an interconnection template determines (i) which composition unit the node belongs to, and (ii) what structural semantics each address field carries. In Figure 4a, before applying ClosLayer at level L1–L2, the address fields are *[Leaf.GPU]*; then a *Spine* field is assigned to each node, yielding *[Spine, Leaf, GPU]*. For example, a GPU node whose address is *1.4.1* is located in the first *Spine* unit (i.e., the whole leaf-spine topology), the fourth *Leaf* unit, and is the first GPU node of the unit. Similarly, in Figure 4b, applying CliqueToR at level L2–L3 adds a *Group* field, yielding *[Group.ToR.GPU]*.

Shortest rule interface. In order to capture shortest routing behaviors under the composition pattern, we denote a routing rule over structured address for a node role r as $\mathsf{F}_r = (\mathsf{M}_r, \mathsf{K}_r)$, where $\mathsf{M}_r(A_{\text{local}}, A_{\text{dst}}) \in \{0, 1\}$ encodes a relative position function, $\mathsf{K}_r(A_{\text{dst}}) \to \kappa$ extracts a key for indexing in the port bucket for the relative position, and A_{local}, A_{dst} represent the local node address and destination node, respectively. For example, in Figure 4a, the *leaf* node has two routing rules. Rule 1 is installed by ClosLayer when composing L0–L1, which means that if the destination node is located within the same *leaf* unit of the *leaf* node, the *GPU* field *d[GPU]* of the destination's address is extracted, and then the key is used as an index in the *down* port bucket to select the egress port *Down-d[GPU]*. If it is not located in the local unit, all ports in the *up* bucket can be used as egress ports to forward up, which is denoted as *U-**.

Notably, Nüwa's interconnection template library is not intended to enumerate every possible AI topology; it targets structured fabrics that satisfy the following requirements:

(1) *Recursive composition.* The topology can be built by recursively composing smaller topology units.
(2) *Deterministic Wiring.* The links introduced at each composition step are deterministic, i.e., they are determined by the positions of the component units and the template parameters.
(3) *Local routing.* Shortest-path next hops can be locally derived from the structural positions of the current node and the destination node.

These requirements are satisfied by many datacenter and AI fabrics, including, but not limited to, Clos/Fat-tree, Dragonfly(+), BCube, dimensional networks (e.g., Torus), HPN, Rail-only, ZCube, and UB-Mesh. Nüwa excludes unstructured or highly irregular networks such as Jellyfish [58] and Xpander-like expanders [59], WANs, and dynamically reconfigurable optical networks. In comparison, htsim [16] currently supports several manually implemented datacenter topologies, including Fat-tree, oversubscribed and multihomed Fat-tree, Dragonfly, BCube, VL2 and Star. We show the intra-host interconnection template in §B.

4.2 Compiler

Nüwa's compiler takes the declarative *topo spec* and realizes it end to end into deployable forwarding artifacts.

Input and artifacts. In the topology specification, the user specifies the concrete parameters for the interconnection templates and the bottom-up application order of the templates. The compiler realizes the spec into four artifacts: (i) an instantiated structured topology instance, (ii) a shortest rule table, (iii) a non-minimal policy table, and (iv) port buckets on each node. Port buckets are generated from the templates' wiring interfaces; they group a node's egress ports by semantic port categories induced by the template wiring.

Compiling a topology specification into artifacts. Following the composition hierarchy specified in the topology specification, the compiler first instantiates topology units through the templates' structural interfaces, completes structured-address assignment, and generates per-node port buckets from the templates' wiring interfaces. It then aggregates the shortest-path rule fragments predefined by templates for each node role and assembles them into the shortest rule table; since different interconnection templates may define different shortest-path rule fragments for the same role, the compiler uses the node role as the key to collect and write them into the final table entries. In the same manner, it aggregates and assembles template-defined non-minimal policy fragments §4.3 into the non-minimal policy table per role. This workflow generates the shortest rule table directly, without global shortest-path computation, and adapts to different structured topologies by changing the template composition in the topo spec.

4.3 Policy-based Non-minimal Routing

In structured topologies, routing policies are common, both for shortest routing and non-minimal routing. In high-throughput,

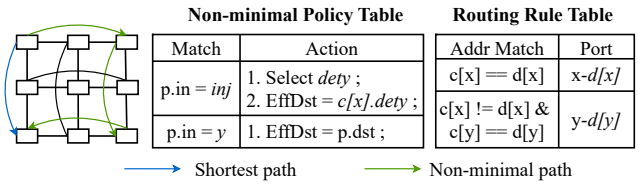


Figure 5: Torus detour path and policy.

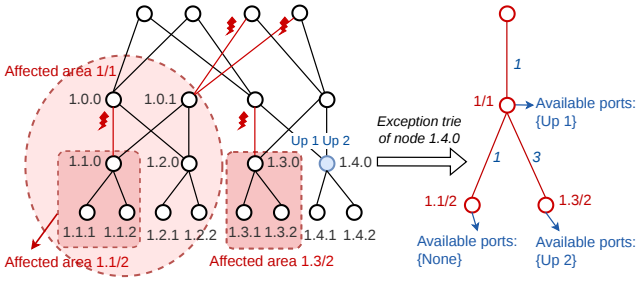


Figure 6: Failures in a Fat-tree. The example considers four failed downward links. For node 1.4.0, they create three affected destination regions highlighted in red: 1/1, 1.1/2, and 1.3/2. Region 1/1 is a coarse affected prefix, and 1.1/2 is a more specific prefix contained within it; 1.3/2 is a separate affected prefix. NÜWA stores these prefixes in the exception trie according to their address prefix hierarchy. A child entry is constrained by its ancestors and any port unavailable for reaching 1/1 is also unavailable for reaching the nested region 1.1/2. The trie therefore captures both containment and independent affected regions, while allowing forwarding to use the most specific matching exception.

low-latency AI training networks with lossless mechanisms (e.g., PFC), routing paths are strictly controlled. Otherwise, cyclic buffer dependency (CBD) may form and cause *PFC deadlock*, which is well known to affect the throughput of the whole network [28]. For example, in Clos topologies, routing paths should follow the *up-and-down* policy, and for multi-dimensional topologies such as 3D-Torus, routing paths should follow *dimension-order-routing (DOR)*. Routing policies are also used to provide feasible non-minimal routing paths to enrich path diversity for load balancing and congestion control.

Therefore, policies can be seen as constraints on feasible ports that are induced by the topology construction pattern. Most policies in structured topologies (such as DOR, Valiant, UGAL, etc.) can be abstracted as selecting a set of waypoints and then routing along the ordered sequence from the source, through the waypoints, to the destination. Within each segment, the packets are still forwarded along shortest paths.

In order to support routing policies, NÜWA needs to (1) identify which stage of the forwarding path the packet is currently in and (2) know what the target node of the current stage is. Accordingly, the packet must carry a routing context consisting of the current effective destination, *effdst*, and an optional *stage*, which indicates how many intermediate nodes have been traversed for policies that may impose a bound on the number of intermediate nodes.

Non-minimal policy interface. We observe that non-minimal detour paths typically occur within the structural semantics defined by a single interconnection template (e.g., Dragonfly's Valiant and UGAL policies occur across different groups under the CLIQUETOR template). If a non-minimal policy needs to span templates, it can also be decomposed across different templates. To capture the intermediate routing semantics, we extend the non-minimal semantic interface at the routing layer of the interconnection template §4.1.1. Concretely, the non-minimal semantic interface is implemented as *<match, action>* rules, which determine (i) the policy for selecting intermediate nodes and (ii) the policy for updating the routing context.

In Figure 5, we show how to implement a detour path that first traverses the *x-axis*, then the *y-axis*, and then returns along the *x-axis* to the destination. The green path indicates this detour, while the blue path indicates the shortest path. There are two rules in the non-minimal policy table for the node role. The first rule specifies that if the packet is injected into the node role for the first time, it can select *dety* as its intermediate target. The second rule means that if the packet reaches the intermediate, it forwards toward the destination along a shortest path (a *dimension-order-routing* path that first traverses the *y-axis*, then the *x-axis*).

4.4 Failure Handling

A link failure (or node failure, modeled as all incident links failing) destroys the symmetric routing rule induced by the interconnection template §4.1. Fortunately, the failed-link rate is typically small, so we can treat failures as exceptions that override the *routing rule matching* process.

Specifically, after a failure, there are two cases: (i) a node still has alternative ports, so it forwards packets through these ports; (ii) a node has no available ports, so the node is either marked as *unreachable* to the destination, or it has a failover policy, which means it can forward packets from *backup* ports. For the first case, reachability is not affected by the failure and is handled by disabling the port. For the second case, the failover policy can be realized through the *policy interface* of the interconnection templates §4.3. So, here we show how to handle the case in which *a failure happens, a node has no other available ports to forward packets.*

Structured topologies are composed by recursively interconnecting topology units. This implies that a failed link will affect the routing behavior toward the whole unit. Therefore, instead of maintaining an exception case for each affected destination node, NÜWA maintains an exception entry *<key, available ports>* for each affected topology unit, indicating available ports for the unit after failure.

There may be several simultaneous failures in the network, and their affected units may overlap. Such overlap follows the recursive topology hierarchy: an exception for a larger unit can contain exceptions for smaller units. These entries are not independent. If a port is unavailable for reaching the larger unit, then the same port cannot be used to reach any smaller unit contained in it. Therefore, the available port set of a smaller-unit exception must be constrained by the available port sets of its covering larger unit exceptions. This containment relation motivates storing exception entries according to their structured address prefixes. NÜWA uses a prefix-masked

structured address as the key. The prefix identifies the affected unit, and the mask length encodes the level of that unit in the recursive composition hierarchy. Such an organization allows NÜWA to find the most specific exception for a destination during forwarding, while also preserving the constraints imposed by larger affected units.

Failure interface. The units that will be affected by a failed link are determined by the structural composition of the topology units, which is exactly captured by the *node rule interface* and the *wiring interface* §4.1.1. So NÜWA implements the exception key generation logic in an additional *failure* interface of the interconnection template. For example, if a link in the CLOSLAYER fails, then on its downward side, *unit-d*, all other units cannot reach *unit-d* through this link; thus the unit *unit-d* is the affected unit, and the exception key should be its unit prefix-masked address. On its upward side, *unit-d* cannot reach any other units through the failed link, so for *unit-d*, the exception is a special wildcard prefix-masked address.

Trie-based exception table. NÜWA maintains an *exception table* on each node, with each affected unit as an entry. Because exception keys are prefix-masked structured addresses, NÜWA stores the failure exception table as a *field-wise prefix trie*, where each level of the trie corresponds to one field of the structured address. The entry records the available port set for destinations matching that key. Since only the affected prefixes are inserted, the trie remains sparse in the common case. At runtime, the exception table exposes two operations: LOOKUP(A_d) and UPDATE(k, C). LOOKUP(A_d) walks the trie along the fields of the destination structured address and returns the entry with the longest matched prefix, if any. The forwarding pipeline in §4.5 consumes this lookup result to decide whether to override the normal rule output. UPDATE maintains consistency among overlapping failures. When inserting or updating an entry for key k, NÜWA first walks the path from the root to the trie node of k and intersects the new available port set with all ancestor entries on this path. It then traverses the subtree rooted at k and intersects descendant entries with the updated entry of k. This ensures that a port unavailable for a larger affected unit is also unavailable for all nested affected units. Figure 6 illustrates a scenario with four downward link failures in Fat-tree.

Local update and withdraw propagation. When a link fails, the switch handles it in four steps.

(1) **Port down.** The switch first marks the corresponding port p as DOWN.
(2) **Generate exception key and update local exception entry.** The switch derives the failure key k for the affected unit and updates the local exception table entry indexed by k:
   (a) Compute the feasible set $C(k)$ associated with key k.
   (b) Remove the failed port: $C(k) \leftarrow C(k) \setminus \{p\}$.
   (c) Insert (or update) the exception entry for k, *and compose it with existing prefix-related entries by intersection.*
(3) **Propagation decision.** If $C(k) \neq \emptyset$, the switch still has an alternative next hop for destinations matching k, and propagation stops. If $C(k) = \emptyset$, the switch is unreachable to the destination region represented by k. To prevent neighbors from selecting it as a next hop for k, the switch broadcasts a *withdraw* message (k, p) to all adjacent neighbors.

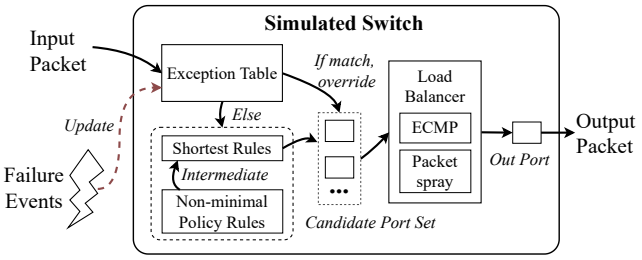


Figure 7: Forwarding pipeline.

(4) **Handling withdraw messages.** Upon receiving a withdraw message (k, p_{in}) from a neighbor over ingress port p_{in}, a node removes the ingress port from the feasible set: $C(k) \leftarrow C(k) \setminus \{p_{\text{in}}\}$, and then inserts or updates the exception entry. Note that p_{in} remains physically available and can still be used for destinations outside the exception key k. If the resulting available set for k becomes empty, the node continues propagation by sending withdraw messages for k to its other adjacent neighbors; otherwise, propagation stops. To prevent redundant propagation, during the handling of a single link-failure event, each node accepts at most one withdraw message per key k from its neighbors.

4.5 Forwarding Pipeline

Figure 7 illustrates NÜWA's per-switch forwarding pipeline. Each simulated switch shares the shortest rule table and non-minimal policy rule table for its node role, and maintains a small exception table that *overrides* the default decision under failures. The forwarding path has three stages: exception override, structured forwarding, and load balancing.

Stage 1: Exception Override Upon receiving a packet, the switch first queries the exception table using the destination structured address A_{dst}:

$$P_{\text{cand}} \leftarrow \texttt{ExceptionTable.Lookup}(A_{\text{dst}}) \tag{1}$$

The lookup performs *longest-prefix match* (LPM) and returns the entry whose key shares the longest common prefix with A_{dst}. If a match is found, the engine directly returns the overridden candidate.

Stage 2: Structured Forwarding If no exception matches, the engine uses routing rules. The engine first matches the destination against each routing rule's match predicate $\mathsf{M}_r(A_{\text{local}}, A_{\text{dst}})$, then the matching rule extracts selected address fields from the packet destination via the key extractor $\mathsf{K}_r(A_{\text{dst}})$ and uses the resulting key to choose a candidate-port bucket:

$$P_{\text{cand}} \leftarrow \mathsf{M}_{r_i}\left[\mathsf{K}_{r_i}(A_{\text{dst}})\right] \quad \text{for the matching rule } i \tag{2}$$

The hierarchical construction ensures that the destination sets covered by different levels of rules at a node are disjoint. Therefore, exactly one rule applies for any packet. When a non-minimal policy is enabled, Stage 2 evaluates the same rule set against an *effective target* derived by the policy rules (e.g., an intermediate target structured address for a detour stage).

Stage 3: Load Balancing If P_{cand} contains multiple ports (e.g., multiple equivalent uplinks), the switch applies a load balancing strategy such as ECMP and selects a final output port $P_{\text{out}} \in P_{\text{cand}}$.

This step is orthogonal to how candidates are derived; it only requires the candidate set.

5 Evaluation

We implemented a prototype of Nüwa based on ns-3 with ~7k LOC of C++. In this section, we evaluate Nüwa. We aim to answer the following research questions:

(1) Does Nüwa preserve routing fidelity?
(2) How much memory does Nüwa cost during simulation?
(3) How fast can Nüwa initialize forwarding artifacts for structured topologies?
(4) How much overhead do failure handling and non-minimal routing policies introduce in Nüwa?

Topologies We parameterize each topology to reflect realistic datacenter configurations. A k-ary Fat-tree supports $k^3/4$ hosts, e.g., 8192 hosts when $k = 32$. For Dragonfly, we adopt full-size configurations ($a = 2p = 2h$, $g = ah + 1$), resulting in $4h^4 + 2h^2$ hosts; for example, 16512 hosts when $h = 8$. Torus configurations use 3D layouts, such as $20 \times 20 \times 20$ for 8,000 hosts. We refer to them as FT (Fat-tree), DF (Dragonfly), and TR (Torus) in the following.

Workloads. We use both synthetic collective traffic and a production trace. For the synthetic collective traffic, we use AllReduce. Unless otherwise specified, each flow carries 1 MB of data per GPU. For realistic traffic, we replay the ATLAHS Grok production training trace collected from a Dragonfly cluster [56]. The trace contains a 2048-GPU workload; in our experiment, we preserve its communication partners and send timing while sampling up to 200 nonzero sends per rank, resulting in 409,600 replayed flows.

Baseline. For network-level simulation, we compare Nüwa with vanilla ns-3 [46] and ns-3-datacenter [12], referred to as ns-3-dc. ns-3-dc is widely used for datacenter and AI-training network evaluation [7–10, 36]. We reimplement ns-3-dc's BFS routing as an ns-3 routing plugin and use the same packet-level simulation components across systems. This setup isolates the effect of the control-plane design within the same ns-3 packet-level simulation framework. We do not use htsim [16] as a primary runtime baseline because htsim differs from ns-3 in simulator architecture, event-processing implementation, data-plane models, and network stack; a direct runtime comparison would therefore make it difficult to attribute the observed performance gains specifically to Nüwa's control-plane optimizations.

Setup. We run experiments on a Linux server with two 32-core Intel(R) Xeon(R) Gold 6338 CPUs @ 2.00GHz and 1T memory.

5.1 Fidelity

Faithful routing behavior is a prerequisite for simulation credibility: if the simulated forwarding decisions deviate from what a real network would produce, performance results (e.g., congestion, tail latency, loss) become untrustworthy. We therefore validate Nüwa's fidelity by comparing its packet-level traces against reference implementations under three representative behaviors.

Shortest routing. We validate shortest-path behavior by comparing Nüwa against a BFS-based shortest routing baseline. For identical traffic inputs, we collect packet traces (per-hop forwarding decisions) from both systems and find them to be exactly identical, confirming that Nüwa reproduces BFS shortest routing.

Failure handling. We randomly inject failures at a rate of 0.001, which is higher than realistic failure rates [70]. Under failures, BFS can produce complex shortest paths (e.g., in a Clos fabric, paths that traverse multiple up–down transitions). Such paths are undesirable in structured topologies because they may increase cyclic dependencies and raise the risk of PFC deadlock [28]. To match practical failure-handling constraints, we restrict the post-failure BFS-recomputed (src, dst) path to be no longer than the pre-failure (src, dst) path. Under this constrained BFS baseline, the packet traces after failures remain exactly identical to Nüwa's traces across the failure scenarios, indicating that Nüwa's exception-based local updates and bounded withdrawals preserve the intended forwarding behavior.

Non-minimal policy. We validate non-minimal routing (Dragonfly Valiant and UGAL routing [32], and a customized 3D-Torus detour routing). For Dragonfly, we handwrite Valiant and UGAL routing logic directly following the original Dragonfly specification [32] as the baseline, and compare it against Nüwa's rule-based Valiant routing by setting the corresponding Valiant policy in the interconnection templates. For our customized 3D-Torus detour routing, we also handwrite the routing logic using dimension-wise offsets based on flow hashes, excluding src/dst and supporting multi-stage detours.

We verified that across identical topology and traffic inputs, the resulting packet traces are exactly identical, demonstrating that Nüwa's non-minimal policy mechanism can faithfully reproduce structured non-minimal routing.

5.2 Memory Footprint

We evaluate the peak memory usage during simulation, as shown in Figure 8. The figure compares Nüwa, ns-3-dc, and ns-3 under the AllReduce workloads across topologies (FT, DF, TR).

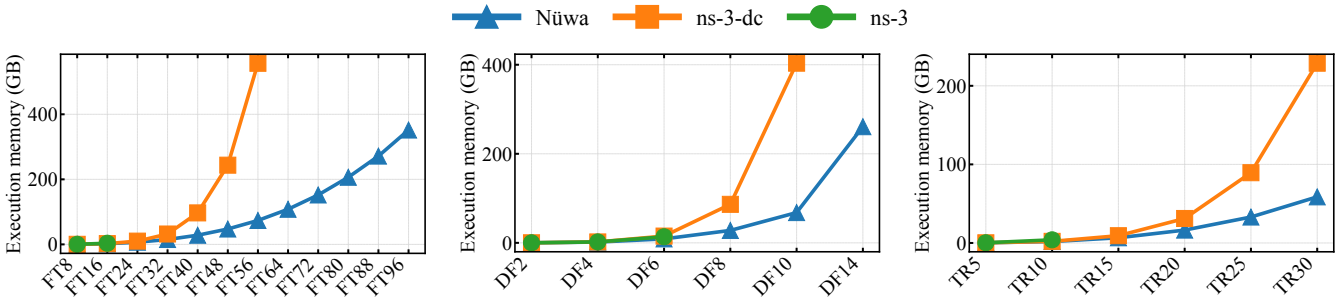
Nüwa consistently exhibits significantly lower memory consumption than ns-3 and ns-3-dc. For example, in FT32, Nüwa uses only 15.0 GB, compared to 31.9 GB for ns-3-dc; at FT48, the gap widens to 47.3 GB vs. 243.6 GB. Even at FT88 (the largest tested), Nüwa completes simulation with ~272 GB, whereas ns-3-dc fails to scale. Similar trends are observed in Dragonfly: at DF10, Nüwa uses 62.4 GB, nearly 6× smaller than ns-3-dc at 398 GB. The memory savings are consistent across all scales.

This makes Nüwa scalable to hyper-scale simulation (FT96 with 220K+ GPUs).

5.3 Simulation Time

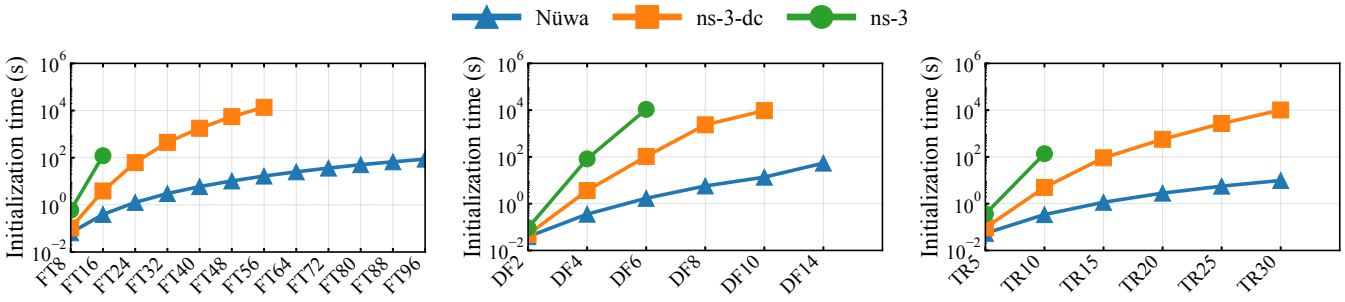
We decompose simulation time into initialization time and execution time. *Initialization time* measures the time spent on topology construction, route-state generation/computation, and other setup tasks before packet events start. *Execution time* measures the time spent on packet-event processing after initialization.

Initialization time. Figure 9 compares initialization times across increasing topology scales, including Fat-tree, Dragonfly, and Torus. Nüwa achieves consistently lower initialization time, demonstrating significantly better scalability than both vanilla ns-3 and ns-3-dc. Specifically, at FT56, Nüwa completes initialization in 20 seconds,



(a) Fat-tree (b) Dragonfly (c) Torus

Figure 8: Memory footprint under different topology scales for three topologies (AllReduce-1MB).



(a) Fat-tree (b) Dragonfly (c) Torus

Figure 9: Initialization time under different topology scales for three topologies.

while ns-3-dc requires over 13,000 seconds and vanilla ns-3 already fails to scale beyond FT32. Even for a 220K-GPU Fat-tree, Nüwa finishes initialization in under 90 seconds. At DF10, Nüwa initializes the topology in under 15 seconds, compared to nearly 10,000 seconds for ns-3-dc. In TR30, Nüwa initializes the topology in under 10 seconds, whereas ns-3-dc exceeds 10,000 seconds and vanilla ns-3 stalls beyond TR10. This performance gain stems from Nüwa's generative design, avoiding explicit per-node route computation.

Execution time. Execution time is dominated by packet-level event processing, including packet transmission, queuing, reception, and forwarding. During *forwarding*, ns-3-dc and vanilla ns-3 need to look up per-destination forwarding tables, which grow rapidly with topology scale and can increase lookup overhead and hurt cache locality. Instead, Nüwa's compact rule tables remain constant in size regardless of topology scale, keeping forwarding efficient as the topology grows. Therefore, although Nüwa primarily optimizes initialization, it also improves execution time.

Figure 10(a)–(c) compare the execution phase simulation time of Nüwa, ns-3-dc, and vanilla ns-3 under the AllReduce workload across three topologies: Fat-tree, Dragonfly, and Torus. Nüwa achieves consistently shorter execution times across all topology scales. In FT16, Nüwa completes execution in 133s, while ns-3-dc takes 136s, and vanilla ns-3 requires over 3,080s. At FT56, Nüwa executes in 5,925s, whereas ns-3-dc takes over 7,181s. Similar trends hold in Dragonfly and Torus: at DF10, Nüwa completes in 17,000s, while ns-3-dc completes in 18,000s, and vanilla ns-3 fails to scale beyond DF6. Nüwa's execution-time advantage becomes more visible as the topology scales.

Figure 11 further evaluates execution time under increasing workload sizes (1–128 MB) on fixed topologies (DF4, DF6, FT16, FT24). Here, Nüwa maintains execution time nearly identical to ns-3-dc across all settings, indicating that its forwarding logic introduces no performance overhead, even under heavy traffic.

Figure 10d shows the total simulation time of Nüwa, ns-3-dc, and vanilla ns-3 under the AllReduce workload on Fat-tree topologies. In FT64, Nüwa completes the entire simulation in 9003 s, which is only 20% of that required by ns-3-dc.

These results collectively demonstrate that Nüwa matches the execution efficiency of optimized simulators like ns-3-dc, while often outperforming them at scale.

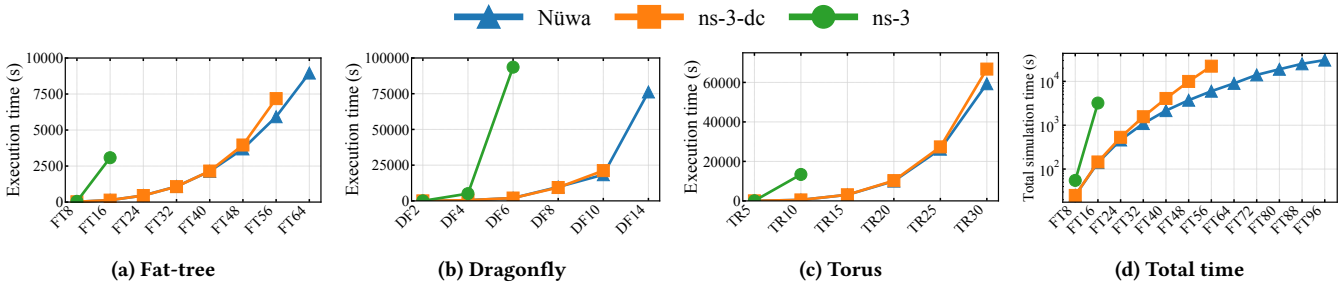
5.4 Production AI workloads

We further evaluate Nüwa on the production Grok trace described earlier and compare it against ns-3-dc. Figure 12 shows that Nüwa reduces initialization memory from 1.74 GB to 0.21 GB, execution memory from 2.07 GB to 0.54 GB, and initialization time from 20.0 s to 0.44 s. Meanwhile, Nüwa and ns-3-dc take 1027 s and 1033 s to complete execution, respectively.

This result validates that Nüwa preserves its memory and time advantages under production traffic: ns-3-dc uses about 4× more memory than Nüwa, and Nüwa is faster than ns-3-dc in both initialization and execution.

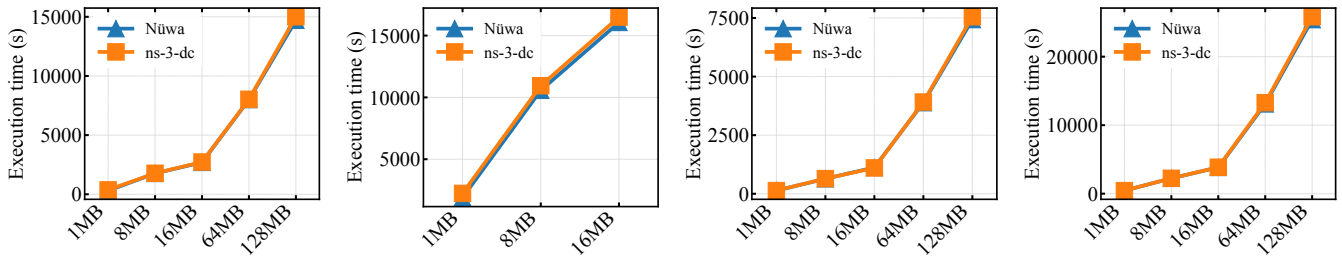
5.5 Failure Handling

We evaluate failure handling on Fat-tree topologies with AllReduce at different scales. We randomly inject failed links at a rate of 0.001



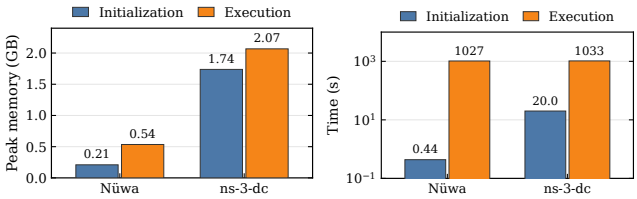
(a) Fat-tree (b) Dragonfly (c) Torus (d) Total time

Figure 10: Simulation time under different topology scales for AllReduce-1MB: (a)–(c) execution time across three topologies and (d) total simulation time in Fat-tree.



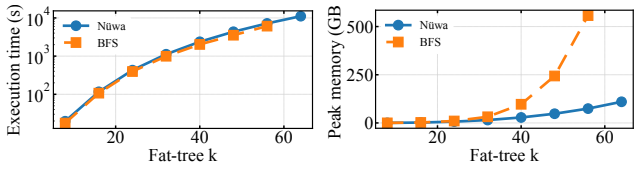
(a) DF4 (b) DF6 (c) FT16 (d) FT24

Figure 11: Execution time under different workload data size.



(a) Peak memory (b) Execution time

Figure 12: Production trace (Grok).



(a) Execution time (b) Peak memory

Figure 13: Failure handling on Fat-tree (AllReduce-1MB).

of the total link count and use the same failure set for the BFS baseline. The BFS baseline is available up to $k = 56$; at $k = 64$ we report Nüwa only.

Failure update time. We measure the time for failure messages to propagate and for all switches to update exception tables, and compare against the baseline that fully recomputes routes via BFS. Nüwa completes failure updates significantly faster than BFS recomputation across all scales (3 s vs. 17,410 s at FT56).

Execution time under failures. We compare the total simulation time under the same failures against BFS with recomputed routes. Nüwa incurs only a small execution-time overhead, indicating efficient failure handling. As shown in Figure 13a, Nüwa's execution time under failures tracks the BFS baseline within a small constant factor across scales (e.g., FT56 is within ~1.2×).

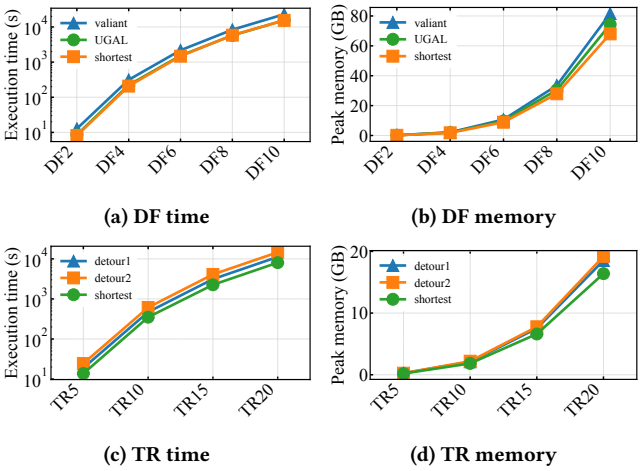
Memory footprint under failures. We report peak simulation memory usage under failures and compare against the BFS baseline. The memory overhead remains small, confirming that failure handling does not inflate runtime state. As shown in Figure 13b, at FT48, Nüwa uses 47.7 GB vs. 243.7 GB for BFS, and at FT56 it uses 74.4 GB vs. 557 GB, indicating a much larger reduction in peak memory and much better scaling under failures.

5.6 Non-minimal Routing

We evaluate our rule-based non-minimal routing on Dragonfly and 3D-Torus. For Dragonfly, we test Valiant and UGAL with $h = 2, 4, 6, 8, 10$. For 3D-Torus detour routing, we use $d = 5, 10, 15, 20$ for $d \times d \times d$ topologies, with the corresponding rule-based shortest routing as the baseline.

Initialization time. We measure the initialization time for each non-minimal routing setting. Initialization time is essentially unchanged by non-minimal policies (within ±2% of baseline across all Dragonfly and Torus settings), since policy state is compiled locally without global recomputation.

Forwarding time. We measure total forwarding time and average per-packet forwarding time during simulation. As shown in Figure 14, execution time overheads are moderate: on Dragonfly, Valiant adds ~1.43–1.56× while UGAL stays near baseline at ~1.03–1.06×; on 3D-Torus, Detour1 incurs ~1.34–1.39× and Detour2 ~1.79×.



(a) DF time (b) DF memory

(c) TR time (d) TR memory

Figure 14: Non-minimal routing overhead (execution time and peak memory).

Memory footprint. We report the peak runtime memory usage under non-minimal routing. As shown in Figure 14, memory overheads remain modest, about 1.08–1.27×, indicating that non-minimal policies increase compute cost more than state.

6 Discussion

Flow level simulation. While Nüwa targets packet-level DES for fidelity, its *generative* control plane is also useful in flow-level simulators. Flow-level models still require a routing substrate to (i) map each flow to a set of feasible paths under ECMP or non-minimal policies, and (ii) remain failure-aware without expensive recomputation. Nüwa can provide this substrate by compiling the same topology spec into compact forwarding semantics and then exposing (a) per-(src,dst) path classes or next-hop sets for probabilistic splitting, and (b) the same exception overlays to keep reachability consistent as failures are injected.

Source routing. Many AI fabrics either already use, or are trending toward, source-routed datapaths (e.g., explicit segment lists or waypoints) to simplify in-network state and enable traffic engineering. Nüwa's design aligns well with this model: it already represents non-minimal routing as waypoint selection plus segment-level shortest-path forwarding, with packets carrying minimal routing context to realize multi-segment detours.

Limitations. Nüwa targets structured fabrics that satisfy recursive composition, deterministic wiring, and local routing over structured addresses. It does not target arbitrary irregular networks, WANs, Jellyfish/Xpander-like expanders, or dynamically reconfigurable optical networks, whose connectivity cannot be compactly captured by the current interconnection-template abstraction.

7 Related Work

Control-plane verification. Prior work on control-plane verification checks network-wide invariants by modeling the control plane induced by routing configurations and protocols. Systems such as Anteater [42], HSA [30], VeriFlow [31], NetPlumber [29], FastPlane [40], and Batfish [23] verify properties such as reachability, loop-freedom, and isolation, and report violations with counterexamples. More recent tools further reason about failures and explore multiple control-plane outcomes (e.g., Minesweeper [15]). These approaches are complementary to Nüwa: they validate correctness of given control-plane configurations, whereas Nüwa focuses on compiling a declarative topology specification into deployable data-plane artifacts.

AI-based network performance estimation. Recent approaches try to use AI-based performance estimation (APE) to avoid the computation overhead of DES. MimicNet [73] keeps a packet-level simulation for a portion of the network while leveraging redundancy and machine learning to approximate other portions of the network to reduce simulation time. DeepQueueNet [69] brings packet-level visibility into APE by combining DES and continuous simulation. Parsimon [74] designs a APE that first simulates each single link and then uses an AI model to predict whole-network performance. m3 [34] uses flow-level fluid simulation to capture essential elements of the path workload. It then feeds this coarse-grained simulation into a machine learning model to estimate network-wide performance. Despite the speedup of AI-based performance estimation, APEs are less accurate than DESs. There is no successful integration of APEs into high-fidelity AI infrastructure simulation yet.

AI-infra simulation. LLMCompass [72] is a hardware evaluation framework for LLM inference workloads. Users can leverage LLMCompass to explore the hardware design space quickly. However, it focuses on the hardware perspective instead of end-to-end system performance simulation, which overlooks network behaviors. ATLAHS [56] is a toolchain designed to accurately generate real-world application workloads, including AI systems. ATLAHS models communication and computation patterns in AI, HPC, and distributed storage applications. It supports multiple network backends, including ns-3 and htsim. ATLAHS is orthogonal to our work and can be used as a traffic generation tool together with Nüwa.

GPU simulation. GPGPU-Sim [14] is a cycle-level simulator for GPGPUs. The popular gem5 simulator also supports cycle-accurate models of AMD GPUs [26]. However, using cycle-accurate GPU simulation for AI infrastructure is overkill for this problem. GPU kernel profiling has been shown to provide enough fidelity [63].

8 Conclusion

This paper presents Nüwa, a generative control plane for AI network simulation that treats routing as a compilation problem. By compiling topology specifications into compact forwarding semantics, Nüwa avoids global shortest-path computation and large routing tables, while supporting non-minimal policies via waypoint constraints and handling failures through prefix-scoped exception overlays with efficient updates. Implemented in ns-3, Nüwa removes the control-plane bottleneck in hyperscale AI fabric simulation and achieves order-of-magnitude improvements in initialization time and memory footprint without sacrificing routing correctness.

Acknowledgments

We thank the anonymous shepherd and reviewers for their valuable comments and helpful feedback. ChatGPT was utilized to assist with text refinement and generation of parts of the code.

References

[1] [n. d.]. CODES: Co-Design of Exascale Storage / Network Simulation Framework. https://wordpress.cels.anl.gov/codes/. Accessed: 2026-02-06.

[2] 2024. Building Meta's GenAI Infrastructure. Engineering at Meta. https://engineering.fb.com/2024/03/12/data-center-engineering/building-metas-genai-infrastructure/ Accessed 2026-02-06.

[3] 2024. Inside the 100K GPU xAI Colossus Cluster (Case Study PDF). Supermicro. https://www.supermicro.com/CaseStudies/Success_Story_xAI_Colossus_Cluster.pdf Accessed 2026-02-06.

[4] 2024. NVIDIA Ethernet Networking Accelerates World's Largest AI Supercomputer, xAI Colossus. NVIDIA Newsroom. https://nvidianews.nvidia.com/news/spectrum-x-ethernet-networking-xai-colossus Accessed 2026-02-06.

[5] 2025. Amazon EKS enables ultra scale AI/ML workloads with support for 100k nodes per cluster. AWS Containers Blog. https://aws.amazon.com/blogs/containers/amazon-eks-enables-ultra-scale-ai-ml-workloads-with-support-for-100k-nodes-per-cluster/ Accessed 2026-02-06.

[6] 2025. Meta's Infrastructure Evolution and the Advent of AI. Engineering at Meta. https://engineering.fb.com/2025/09/29/data-infrastructure/metas-infrastructure-evolution-and-the-advent-of-ai/ Accessed 2026-02-06.

[7] Vamsi Addanki, Maria Apostolaki, Manya Ghobadi, Stefan Schmid, and Laurent Vanbever. 2022. ABM: Active buffer management in datacenters. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 36–52.

[8] Vamsi Addanki, Wei Bai, Stefan Schmid, and Maria Apostolaki. 2024. Reverie: Low pass {Filter-Based} switch buffer sharing for datacenters with {RDMA} and {TCP} traffic. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 651–668.

[9] Vamsi Addanki, Oliver Michel, and Stefan Schmid. 2022. {PowerTCP}: Pushing the performance limits of datacenter networks. In *19th USENIX symposium on networked systems design and implementation (NSDI 22)*. 51–70.

[10] Vamsi Addanki, Maciej Pacut, and Stefan Schmid. 2024. Credence: Augmenting datacenter switch buffer sharing with {ML} predictions. In *21st USENIX symposium on networked systems design and implementation (NSDI 24)*. 613–634.

[11] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. 2008. A Scalable, Commodity Data Center Network Architecture. In *Proceedings of the ACM SIGCOMM 2008 Conference*. 63–74.

[12] INET at TU Berlin. 2022. ns-3-datacenter. https://github.com/inet-tub/ns3-datacenter.

[13] Songyuan Bai, Hao Zheng, Chen Tian, Xiaoliang Wang, Chang Liu, Xin Jin, Fu Xiao, Qiao Xiang, Wanchun Dou, and Guihai Chen. 2024. Unison: A Parallel-Efficient and User-Transparent Network Simulation Kernel. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*. 115–131.

[14] Ali Bakhoda, George L Yuan, Wilson WL Fung, Henry Wong, and Tor M Aamodt. 2009. Analyzing CUDA Workloads Using a Detailed GPU Simulator. In *2009 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 163–174.

[15] Ryan Beckett, Aarti Gupta, Ratul Mahajan, and David Walker. 2017. A general approach to network configuration verification. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 155–168.

[16] Broadcom. 2025. htsim Network Simulator. https://github.com/Broadcom/csg-htsim.

[17] Jiamin Cao, Yu Guan, Kun Qian, Jiaqi Gao, Wencong Xiao, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. 2024. Crux: Gpu-efficient communication scheduling for deep learning training. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 1–15.

[18] Peirui Cao, Wenxue Cheng, Shizhen Zhao, and Yongqiang Xiong. 2024. Network load balancing with parallel flowlets for AI training clusters. In *Proceedings of the 2024 SIGCOMM Workshop on Networks for AI Computing*. 18–25.

[19] Google Cloud. 2025. VM instance pricing | Google Cloud. https://cloud.google.com/compute/vm-instance-pricing?hl=en#section-5.

[20] E. W. Dijkstra. 1959. A Note on Two Problems in Connexion with Graphs. *Numer. Math.* 1, 1 (1959), 269–271. doi:10.1007/BF01386390

[21] David Eppstein. 1998. Finding the k Shortest Paths. *SIAM J. Comput.* 28, 2 (1998), 652–673. doi:10.1137/S0097539795290477

[22] Yicheng Feng, Yuetao Chen, Kaiwen Chen, Jingzong Li, Tianyuan Wu, Peng Cheng, Chuan Wu, Wei Wang, Tsung-Yi Ho, and Hong Xu. 2024. Echo: Simulating distributed training at scale. *arXiv preprint arXiv:2412.12487* (2024).

[23] Ari Fogel, Stanley Fung, Luis Pedrosa, Meg Walraed-Sullivan, Ramesh Govindan, Ratul Mahajan, and Todd Millstein. 2015. AGeneral Approach to Network Configuration Analysis. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. 469–483.

[24] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, et al. 2024. RDMA over Ethernet for Distributed Training at Meta Scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 57–70.

[25] Kaihui Gao, Li Chen, Dan Li, Vincent Liu, Xizheng Wang, Ran Zhang, and Lu Lu. 2023. Dons: Fast and affordable discrete event network simulation with automatic parallelization. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 167–181.

[26] gem5. 2025. gem5: Full System AMD GPU model. https://www.gem5.org/documentation/general_docs/gpu_models/gpufs.

[27] Fei Gui, Kaihui Gao, Li Chen, Dan Li, Vincent Liu, Ran Zhang, Hongbing Yang, and Dian Xiong. 2025. Accelerating Design Space Exploration for LLM Training Systems with Multi-experiment Parallel Simulation. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 473–488.

[28] Shuihai Hu, Yibo Zhu, Peng Cheng, Chuanxiong Guo, Kun Tan, Jitendra Padhye, and Kai Chen. 2017. Tagger: Practical PFC Deadlock Prevention in Data Center Networks. In *Proceedings of the 13th International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*. 451–463.

[29] Peyman Kazemian, Michael Chang, Hongyi Zeng, George Varghese, Nick McKeown, and Scott Whyte. 2013. Real time network policy checking using header space analysis. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*. 99–111.

[30] Peyman Kazemian, George Varghese, and Nick McKeown. 2012. Header space analysis: Static checking for networks. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*. 113–126.

[31] Ahmed Khurshid, Wenxuan Zhou, Matthew Caesar, and P Brighten Godfrey. 2012. Veriflow: Verifying network-wide invariants in real time. In *Proceedings of the first workshop on Hot topics in software defined networks*. 49–54.

[32] John Kim, Wiliam J Dally, Steve Scott, and Dennis Abts. 2008. Technology-Driven, Highly-Scalable Dragonfly Topology. In *2008 ACM/IEEE 35th International Symposium on Computer Architecture (ISCA)*. 77–88.

[33] Sandia National Laboratories. 2025. SST: Structural Simulation Toolkit. https://sst-simulator.org.

[34] Chenning Li, Arash Nasr-Esfahany, Kevin Zhao, Kimia Noorbakhsh, Prateesh Goyal, Mohammad Alizadeh, and Thomas E Anderson. 2024. m3: Accurate Flow-Level Performance Estimation using Machine Learning. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 813–827.

[35] Wenkai Li, Ran Shu, Peng Zhang, and Yongqiang Xiong. 2025. Nüwa: Efficient Generative Control Plane for AI Network Simulation. In *Proceedings of the 9th Asia-Pacific Workshop on Networking (APNet)*. 121–127.

[36] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, et al. 2019. HPCC: High Precision Congestion Control. In *Proceedings of the ACM SIGCOMM 2019 Conference*. 44–58.

[37] Jinkun Lin, Ziheng Jiang, Zuquan Song, Sida Zhao, Menghan Yu, Zhanghan Wang, Chenyuan Wang, Zuocheng Shi, Xiang Shi, Wei Jia, et al. 2025. Understanding Stragglers in Large Model Training Using What-if Analysis. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 483–498.

[38] Ning Liu, Adnan Haider, Xian-He Sun, and Dong Jin. 2015. Fattreesim: Modeling large-scale fat-tree networks for hpc systems and data centers using parallel and discrete event simulation. In *Proceedings of the 3rd ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*. 199–210.

[39] Fei Long, Kaihui Gao, Li Chen, Dan Li, Yiwei Zhang, Fei Gui, Yitao Xing, Wenjia Wei, and Bingyang Liu. 2026. Supercharging Packet-level Network Simulation of Large Model Training via Memoization and {Fast-Forwarding}. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. 1131–1151.

[40] Nuno P Lopes and Andrey Rybalchenko. 2019. Fast BGP Simulation of Large Datacenters. In *Proceedings of 20th International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI 2019)*. 386–408.

[41] OpenSim Ltd. 2024. OMNeT++ Discrete Event Simulator. https://omnetpp.org.

[42] Haohui Mai, Ahmed Khurshid, Rachit Agarwal, Matthew Caesar, P Brighten Godfrey, and Samuel Talmadge King. 2011. Debugging the data plane with anteater. *ACM SIGCOMM Computer Communication Review* 41, 4 (2011), 290–301.

[43] Sarah McClure, Sylvia Ratnasamy, and Scott Shenker. 2025. Load Balancing for AI Training Workloads. *arXiv preprint arXiv:2507.21372* (2025).

[44] Qingkai Meng, Hao Zheng, Zhenhui Zhang, ChonLam Lao, Chengyuan Huang, Baojia Li, Ziyuan Zhu, Hao Lu, Weizhen Dang, Zitong Lin, et al. 2025. Astral: A Datacenter Infrastructure for Large Language Model Training at Scale. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 609–625.

[45] John Moy. 1998. OSPF Version 2. RFC 2328. doi:10.17487/RFC2328

[46] nsnam. 2024. ns-3 | a discrete-event network simulator for internet systems. https://www.nsnam.org.

[47] Dylan Patel and Daniel Nishball. 2024. 100k H100 Clusters: Power, Network Topology, Ethernet vs InfiniBand, Reliability, Failures, Checkpointing. https://www.semianalysis.com/p/100000-h100-clusters-power-network.

[48] Chenchen Qi, Wenfei Wu, Yongcan Wang, Keqiang He, Yu-Hsiang Kao, Zongying He, Chen-Yu Yen, Zhuo Jiang, Feng Luo, Surendra Anubolu, et al. 2025. SGLB: Scalable and Robust Global Load Balancing in Commodity AI Clusters. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 626–644.

[49] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, et al. 2024. Alibaba HPN: A Data Center Network for Large Language Model Training. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 691–706.

[50] Yicheng Qian, Ran Shu, Rui Ma, Yang Wang, Derek Chiou, Nadeen Gebara, Luca Piccolboni, Miriam Leeser, and Yongqiang Xiong. 2025. Miniature: Fast AI Supercomputer Networks Simulation on FPGAs. In *Proceedings of the 9th Asia-Pacific Workshop on Networking (APNet)*. 114–120.

[51] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. 2024. {CASSINI}:{Network-Aware} job scheduling in machine learning clusters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1403–1420.

[52] Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2020. ASTRA-SIM: Enabling SW/HW Co-Design Exploration for Distributed DL Training Platforms. In *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 81–92.

[53] Yakov Rekhter, Susan Hares, and Tony Li. 2006. A Border Gateway Protocol 4 (BGP-4). RFC 4271. doi:10.17487/RFC4271

[54] Duncan Roweth, Greg Faanes, Marten Terpstra, and Jesse Treger. 2022. Hpe slingshot launched into network space. *Proceedings of the Cray User Group* (2022).

[55] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. {TACCL}: Guiding collective algorithm synthesis using communication sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 593–612.

[56] Siyuan Shen, Tommaso Bonato, Zhiyi Hu, Pasquale Jordan, Tiancheng Chen, and Torsten Hoefler. 2025. ATLAHS: An Application-centric Network Simulator Toolchain for AI, HPC, and Distributed Storage. In *SC'25: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 349–367.

[57] Arjun Singh, Joon Ong, Amit Agarwal, Glen Anderson, Ashby Armistead, Roy Bannon, Seb Boving, Gaurav Desai, Bob Felderman, Paulie Germano, et al. 2015. Jupiter Rising: A Decade of Clos Topologies and Centralized Control in Google's Datacenter Network. In *Proceedings of the ACM SIGCOMM 2015 Conference*. 183–197.

[58] Ankit Singla, Chi-Yao Hong, Lucian Popa, and P Brighten Godfrey. 2012. Jellyfish: Networking data centers randomly. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*. 225–238.

[59] Asaf Valadarsky, Gal Shahaf, Michael Dinitz, and Michael Schapira. 2016. Xpander: Towards optimal-performance datacenters. In *Proceedings of the 12th International on Conference on emerging Networking EXperiments and Technologies*. 205–219.

[60] Dan Wang, Peng Zhang, Wenbing Sun, Wenkai Li, Xing Feng, Hao Li, Jiawei Chen, Weirong Jiang, and Yongping Tang. 2025. S2: A Distributed Configuration Verifier for Hyper-Scale Networks. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 796–808.

[61] Weiyang Wang, Manya Ghobadi, Kayvon Shakeri, Ying Zhang, and Naader Hasani. 2024. Rail-only: A Low-Cost High-Performance Network for Training LLMs with Trillion Parameters. In *2024 IEEE Symposium on High-Performance Interconnects (HOTI)*. 1–10.

[62] Weiyang Wang, Moein Khazraee, Zhizhen Zhong, Manya Ghobadi, Zhihao Jia, Dheevatsa Mudigere, Ying Zhang, and Anthony Kewitsch. 2023. {TopoOpt}: Co-optimizing network topology and parallelization strategy for distributed training jobs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 739–767.

[63] Xizheng Wang, Qingxu Li, Yichi Xu, Gang Lu, Dan Li, Li Chen, Heyang Zhou, Linkang Zheng, Sen Zhang, Yikai Zhu, et al. 2025. SimAI: Unifying Architecture Design and Performance Tuning for Large-Scale Large Language Model Training with Scalability and Precision. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 541–558.

[64] Zhaodong Wang, Satyajeet Singh Ahuja, Xu Zhang, Max Noormohammadpour, Gregory R Steinbrecher, Thomas Fuller, Xin Liu, Kevin Quirk, Mikel Jimenez Fernandez, Abhinav Triguna, et al. 2026. Enabling {AI} Network {Cross-Layer} Design and Operations with Arcadia: A Simulation Platform at Scale. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. 1791–1807.

[65] William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2023. ASTRA-sim2.0: Modeling Hierarchical Networks and Disaggregated Systems for Large-model Training at Scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 283–294.

[66] xAI. 2025. Grok 4 | xAI. https://x.ai/news/grok-4.

[67] Zihan Yan, Dan Li, Li Chen, Dian Xiong, Kaihui Gao, Yiwei Zhang, Rui Yan, Menglei Zhang, Bochun Zhang, Zhuo Jiang, et al. 2025. From ATOP to ZCube: Automated topology optimization pipeline and a highly cost-effective network topology for large model training. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 861–881.

[68] Zihan Yan, Dan Li, Li Chen, Dian Xiong, Kaihui Gao, Yiwei Zhang, Rui Yan, Menglei Zhang, Bochun Zhang, Zhuo Jiang, et al. 2025. From ATOP to ZCube: Automated Topology Optimization Pipeline and A Highly Cost-Effective Network Topology for Large Model Training. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 861–881.

[69] Qingqing Yang, Xi Peng, Li Chen, Libin Liu, Jingze Zhang, Hong Xu, Baochun Li, and Gong Zhang. 2022. DeepQueueNet: Towards Scalable and Generalized Network Performance Estimation with Packet-level Visibility. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 441–457.

[70] Zhiyi Yao, Pengbo Hu, Congcong Miao, Xuya Jia, Zuning Liang, Yuedong Xu, Chunzhi He, Hao Lu, Mingzhuo Chen, Xiang Li, et al. 2025. Holmes: Localizing irregularities in {LLM} training with mega-scale {GPU} clusters. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 523–540.

[71] Jin Y. Yen. 1971. Finding the K Shortest Loopless Paths in a Network. *Management Science* 17, 11 (July 1971), 712–716. doi:10.1287/mnsc.17.11.712

[72] Hengrui Zhang, August Ning, Rohan Baskar Prabhakar, and David Wentzlaff. 2024. LLMCompass: Enabling Efficient Hardware Design for Large Language Model Inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 1080–1096.

[73] Qizhen Zhang, Kelvin KW Ng, Charles Kazer, Shen Yan, João Sedoc, and Vincent Liu. 2021. MimicNet: Fast Performance Estimates for Data Center Networks with Machine Learning. In *Proceedings of the ACM SIGCOMM 2021 Conference*. 287–304.

[74] Kevin Zhao, Prateesh Goyal, Mohammad Alizadeh, and Thomas E Anderson. 2023. Scalable Tail Latency Estimation for Data Center Networks. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 685–702.

Appendices

Appendices are supporting material that has not been peer-reviewed.

A Interconnection Template Illustration

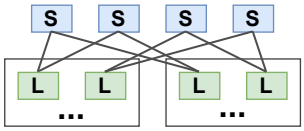
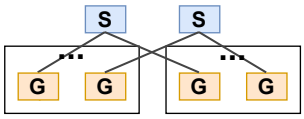
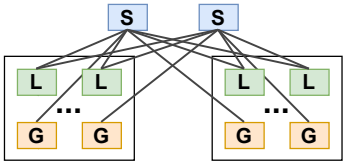
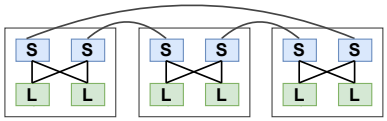
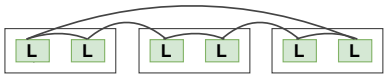
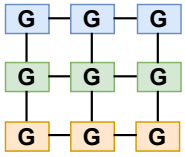
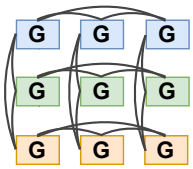
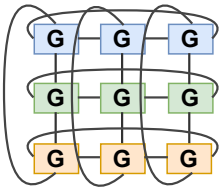
Rule	Schematic	Key params	Used in
ClosLayer		`group_num`	Fat-tree, Clos, Rail-Optimized, HPN, UB-Mesh, DragonFly(+)
BCubeLayer		–	BCube, ZCube
ZCubeLayer		–	ZCube
CliqueLF		`mode` ∈ {`abs`, `rel`}	DragonFly+
CliqueToR		`mode` ∈ {`abs`, `rel`}	Dragonfly
DimMesh		`dim`	Mesh
DimFullMesh		`dim`	FullMesh, UB-Mesh
DimTorus		`dim`	Torus

Table 3: Illustrated reference for the templates in Nüwa's built-in interconnection template library (shown in Table 2). Each schematic shows a small example instance of the wiring pattern enumerated by the rule and is provided for illustration only (not to scale).

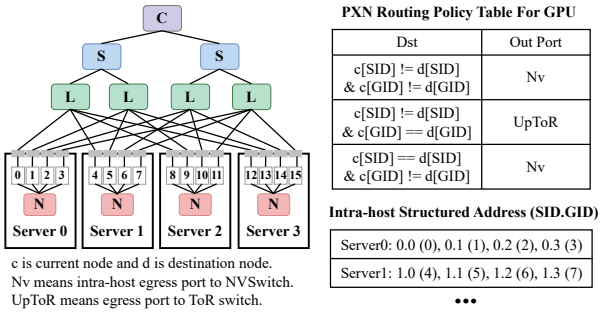


Figure 15: Rail-optimized topology and PXN routing policy.

B Intra-host Network

AI training clusters commonly attach multiple GPUs to a single server and rely on an intra-host fabric (e.g., NVLink, NVSwitch, and PCIe) for high-bandwidth communication. In practice, GPU communication stacks expose topology-aware mechanisms that can forward traffic through intermediate GPUs within a host and can steer inter-node traffic to a non-local NIC via an intermediate GPU (e.g., PXN). NüWA models this intra-host fabric explicitly so that forwarding choices remain faithful to the local control-plane logic, rather than treating a host as a black box.

Construction via an IntraHost interconnection template. NüWA introduces a new IntraHost interconnection template for intra-host networking. After the inter-host topology is constructed, NüWA declares host membership for leaf GPUs (i.e., which GPUs belong to the same host). For each host, NüWA instantiates an IntraHost interconnection template that connects GPUs within the host, optionally via newly created NVSwitch nodes when the configuration includes NVSwitch. The IntraHost rule is compiled in the same way as inter-host rules: it deterministically enumerates intra-host links and synthesizes per-node mapping tables used by forwarding (§4.5).

Intra-host addressing. Each GPU is assigned an intra-host structured address of the form `SID.GID`, where `SID` identifies the server and `GID` identifies the GPU within that server. This is a *separate* address namespace from the inter-host structured address. It is used only for intra-host policy matching and is not merged into the global structured address. Intra-host matching takes priority before inter-host forwarding.

Intra-host routing policies. NüWA installs an intra-host policy table on GPU nodes (and NVSwitch nodes, if present) keyed by the intra-host address. This table has higher priority than inter-host forwarding.

NüWA implements intra-host routing policies at the IntraHost rule. We support three mechanisms: (i) direct intra-host peer-to-peer forwarding over the intra-host fabric, (ii) intra-node forwarding through an intermediate GPU to exploit NVLink connectivity when a direct path is undesirable, and (iii) PXN-style egress steering, where inter-node traffic can be forwarded to a non-local NIC using NVLink and an intermediate GPU before leaving the host. These mechanisms are represented as local-match actions over `SID.GID` and ingress/egress port roles, keeping intra-host state compact and preserving the same forwarding pipeline abstraction (§4.5).

Figure 15 illustrates a rail-optimized setup where each server hosts 4 GPUs via an NVSwitch. Each GPU is assigned an intra-host structured address of the form `SID, GID`, where `SID` identifies the server ID and `GID` identifies the GPU ID (e.g., `1.2` denotes GPU 2 in server 1). To model PXN-style egress steering, NüWA applies a routing rule over these intra-host fields: when the destination resides on a different server ($c[\text{SID}] \neq d[\text{SID}]$) but shares the same GPU index ($c[\text{GID}] = d[\text{GID}]$), the packet is forwarded to the NVSwitch and then to the local GPU from which it exits the host. This rule is keyed by structured-address fields rather than enumerating destinations; it remains compact while still capturing topology-aware host networking behavior.